

Lab-19.2

Course : AI Assisted Coding

Topic: Code Translation: Converting Between Programming Languages

Name : Mohammed Sabir

Hall Ticket No : 2303A51506

Batch : 22

Task-01

Final Optimal Prompt:

Convert the following Python file handling program into an equivalent Java program. Ensure proper file reading/writing, exception handling, and input/output formatting. The Java program should produce the same output as the Python code.

<Insert Python file handling code here>

Code Screenshot :

lab17.py lab17.py\...

```
1 with open("data1.txt", "r") as file:
2     content = file.read()
3     print(content)
```

FileReadExample.java FileReadExample.java FileReadExample

```
1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.io.IOException;
4
5 public class FileReadExample {
6     Run | Debug
7     public static void main(String[] args) {
8         String fileName = "data1.txt";
9
10        try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
11            String line;
12
13            while ((line = reader.readLine()) != null) {
14                System.out.println(line);
15            }
16        } catch (IOException e) {
17            System.out.println("Error reading file: " + e.getMessage());
18        }
19    }
20 }
```

```
3 #     print(content)
4 with open("data1.txt", "w") as file:
5     file.write("Hello World")
```

```

1  import java.io.FileWriter;
2  import java.io.IOException;
3
4  public class FileWriteExample {
5      Run | Debug
6      public static void main(String[] args) {
7          try (FileWriter writer = new FileWriter("data2.txt")) {
8              writer.write("Hello World");
9          } catch (IOException e) {
10             System.out.println("Error writing file: " + e.getMessage());
11         }
12     }
13 }

```

Output Screenshot:

```

● PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant> & 'c:\python314\2-x64\bundled\libs\debugpy\launcher' '63797' '--' 'c:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant\src\Main.java'
Hello Sabir,
Welcome to the Student Management System.
This file contains basic information.
Have a great day!
○ PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant>

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE
● PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant> & 'C:\Program Files\Microsoft\jdk-17.0.18-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\sakir\AppData\Roaming\Code\User\workspaceStorage\44713958e33907037c4f923d0b08b7bd\redhat.java\jdt_ws\Ai-Assistant_12260612\bin' 'FileWriteExample'
Hello Sabir,
Welcome to the Student Management System.
This file contains basic information.
Have a great day!
○ PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant>

```

```

≡ data1.txt data1.txt
1  Hello World

```

```

≡ data2.txt data2.txt
1  Hello World

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

This task demonstrates how Python file handling operations can be translated into Java while maintaining the same functionality. Python uses a simple and concise approach with the `with open()` statement, which automatically handles file closing. In contrast, Java requires explicit classes such as `FileReader`, `BufferedReader`, and `FileWriter` along with `try-with-resources` for proper resource management. Exception handling is mandatory in Java, making the code more robust but slightly more verbose. Despite these differences, both implementations produce the same output, highlighting how similar logic can be implemented across different programming languages.

Task-02**Final Optimal Prompt:**

Convert the following C++ sorting algorithm into an equivalent Python program. Ensure the logic remains the same and the output is correct for different test cases.

<Insert C++ sorting code here>

Code Screenshot :

lab17.c++ lab17.c++\main()

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int arr[] = {5, 3, 8, 4, 2};
6      int n = 5;
7
8      for(int i = 0; i < n-1; i++) {
9          for(int j = 0; j < n-i-1; j++) {
10             if(arr[j] > arr[j+1]) {
11                 int temp = arr[j];
12                 arr[j] = arr[j+1];
13                 arr[j+1] = temp;
14             }
15         }
16     }
17
18     for(int i = 0; i < n; i++) {
19         cout << arr[i] << " ";
20     }
21
22     return 0;
23 }
```

```
8  arr = [5, 3, 8, 4, 2]
9  n = len(arr)
10
11  for i in range(n - 1):
12      for j in range(n - i - 1):
13          if arr[j] > arr[j + 1]:
14              # swap
15              arr[j], arr[j + 1] = arr[j + 1], arr[j]
16
17  print("Sorted array:", arr)
```

Output Screenshot:

```
xe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bund
yr 2sem\Ai-Assistant\lab17.py'
Sorted array: [2, 3, 4, 5, 8]
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant> █
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

This task demonstrates how a sorting algorithm written in C++ can be effectively translated into Python while preserving the original logic. The structure of loops and comparison conditions remains the same, but Python simplifies syntax by removing explicit type declarations and using built-in features like tuple swapping. The Python version is more concise and readable, making it easier to understand and maintain. Despite these differences, both implementations produce the same output for different test cases, ensuring correctness. This highlights Python's simplicity and flexibility in implementing standard algorithms efficiently.

Task-03

Final Optimal Prompt:

Code Screenshot :

```
1  -- SQLite
2
3  SELECT department, COUNT(*) AS employee_count, AVG(salary) AS average_salary
4  FROM employees
5  GROUP BY department;
6
```

```

# Pandas Implementation
import pandas as pd
# Sample dataset
data = {
    'department': ['HR', 'IT', 'Finance', 'IT', 'HR', 'Finance'],
    'salary': [50000, 80000, 60000, 90000, 55000, 65000]
}
df = pd.DataFrame(data)
# Group by department and calculate aggregate functions
result = df.groupby('department').agg({
    'salary': ['count', 'mean']
}).reset_index()
print(result)

```

Output Screenshot:

	department	salary	
		count	mean
0	Finance	2	62500.0
1	HR	2	52500.0
2	IT	2	85000.0

	department	salary	
		count	mean
0	Finance	2	62500.0
1	HR	2	52500.0
2	IT	2	85000.0

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

This task demonstrates how SQL aggregation operations can be effectively translated into Pandas DataFrame operations in Python. The GROUP BY clause in SQL corresponds to the groupby() function in Pandas, while aggregate functions like COUNT are implemented using methods such as count() or size(). Pandas provides a flexible and powerful way to manipulate and analyze data programmatically. The conversion maintains the same logic and output while offering additional capabilities for data processing. This highlights how Python can be used as an alternative to SQL for performing data analysis tasks efficiently.

Task-04

Final Optimal Prompt:

Convert the following code from one programming language to another (e.g., Python to Java / Python to C++). Ensure both versions have the same functionality. Also explain key differences in syntax, libraries, and execution.

<Insert source code here>

Code Screenshot :

```
19
20 def calculate_sum(a, b):
21     return a + b
22
23 num1 = 5
24 num2 = 10
25
26 result = calculate_sum(num1, num2)
27 print("Sum:", result)
```

```
34 public class Main {
35
36     public static int calculateSum(int a, int b) {
37         return a + b;
38     }
39
40     Run | Debug
41     public static void main(String[] args) {
42         int num1 = 5;
43         int num2 = 10;
44
45         int result = calculateSum(num1, num2);
46         System.out.println("Sum: " + result);
47     }
48 }
```


Output Screenshot:

```
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant> c;; cd 'c:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant\lab17.py'
Sum: 15
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant>
```

```
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant> & 'C:\Program Files\Microsoft  
ionMessages' '-cp' 'C:\Users\sakir\AppData\Roaming\Code\User\workspaceStorage\4713958e339070  
n'  
Sum: 15  
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

This task demonstrates how code can be translated between programming languages while maintaining the same functionality. Although the logic remains identical, the syntax and structure differ significantly. Python offers a concise and flexible syntax with dynamic typing, making it easier to write and understand. In contrast, Java requires explicit data types and follows a strict class-based structure, which enhances type safety and scalability. The conversion highlights the importance of understanding language-specific features, execution models, and standard libraries to ensure accurate and efficient implementation across different programming environments.

Tack-05

Final Optimal Prompt:

Convert the following Python function into an equivalent C++ program. Ensure correct syntax, input handling, and output formatting. The program should correctly identify whether a number is even or odd.

<Insert Python function here>

Code Screenshot :

```
28
29 def check_even_odd(num):
30     if num % 2 == 0:
31         print("Even")
32     else:
33         print("Odd")
34
35 num = int(input("Enter a number: "))
36 check_even_odd(num)
```



The screenshot shows a C++ code editor with a file named 'main.cpp'. The code defines a function 'checkEvenOdd' that takes an integer 'num' and prints 'Even' or 'Odd' based on whether 'num' is divisible by 2. The 'main' function prompts the user to enter a number and calls the 'checkEvenOdd' function.

```
main.cpp
1  #include <iostream>
2  using namespace std;
3
4  // Function to check even or odd
5  void checkEvenOdd(int num) {
6      if (num % 2 == 0) {
7          cout << "Even" << endl;
8      } else {
9          cout << "Odd" << endl;
10     }
11 }
12
13 int main() {
14     int num;
15
16     cout << "Enter a number: ";
17     cin >> num;
18
19     checkEvenOdd(num);
20
21     return 0;
22 }
```

Output Screenshot:

```
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant> c++; cd 'c:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant\lab17.py'
Enter a number: 16
Even
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant> |
```

Output

```
Enter a number: 24
Even
```

```
=== Code Execution Successful ===
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

This task demonstrates how a simple Python function can be translated into C++ while preserving its core logic. The even or odd check remains identical, but the syntax differs due to language requirements. Python uses a concise and flexible structure with dynamic typing, whereas C++ requires explicit data types, function definitions, and a structured main function. Input and output handling also differ, with Python using `input()` and `print()`, while C++ uses `cin` and `cout`. This conversion highlights the importance of understanding syntax and execution differences between programming languages.